Deliverable 1: Documentation

Autors

Pol Ortega Fernàndez: 49254483Y

Andreu Puig-gròs Munté: 48255353L

Marc Pujol Navajo: 49259109D

Escola Politècnica Superior

Work presented to: Roberto Garcia and David Sarrat

Subject: Web project

Course: 2025-2026

Date of delivery: 19 of April, 2026

Contents

Introduction **1**
1. Github public address **2**
1.1. Project Source Code & Open Source Repository 2
1.2. Accessing the Repository 2
1.3. What can be found in the repository 2
1.4. Development 2
2. Notes about all the design decisions **3**
2.1. Database schema 3
2.2. Admin interface 5
2.3. User authentication and registration 5
2.4. Docker orchestration using docker-compose 6
2.5. Adhere to the 12-factor guidelines 7
2.6. Web pages to display and allow browsing application data 8
3. Distribution of marks **10**

Introduction

The following report provides a comprehensive yet simple technical breakdown of the development process for the Django-based web application presented in this subject. It's designed to facilitate the access and data management of a competitive Counter-Strike 1.6 related website that is exclusively focused on 1v1 game matches inside the web browser along ranking and user customization features.

This project was developed as a direct evolution of the initial proposal made by the professors' team. It incorporates the given constructive feedback and technical requirements to ensure a robust final product. The current deliverable implements a sophisticated relational database model designed to handle complex relation hierarchies as well as match history, player rankings amongst other traits. This ecosystem is managed through a mildly customized administrative interface and a secure user authentication system that extends the django base, and prioritizes data integrity and session safety.

In order to ensure the high standards of software portability and scalability, the app is containerized using Docker and orchestrated via Docker Compose. Moreover, by following the well-known 12-Factor App methodology, the project ensures a clean separation between code and configuration, facilitating the development workflow. This documentation provides an overview of our proposed solution, and it includes the revised data model documentation, along with our database models' implementation design, (satisfying the given professors' feedback), all the design decisions, the detailed instructions on how to run/deploy our application, and some interesting custom implementations, amongst others.

1. Github public address

1.1. Project Source Code & Open Source Repository

The development of the Strike 1v1 project is publicly accessible on the GitHub platform. By doing it this way, we can allow everyone to see the main web page's source code as well as, once finished, the implementation of other features such as matchmaking and the ability to play through the website. Also, by hosting our source code on GitHub, we can also let anyone interested review our multiple open issues, pull requests and all the program logic. They will also be able to track the whole evolution of the platform from its alpha stages (preliminary) to the production environment (advanced).

1.2. Accessing the Repository

The GitHub public address for the project's source code is the following one:

- **GitHub URL:** https://github.com/MarcPujolUNI/Strike1v1

Also, we provide the link to the Strike 1v1 server, where we are developing the server to play:

- **GitHub URL:** https://github.com/Apm-96/1v1Strike-server

1.3. What can be found in the repository

The core engine of our website has been implemented using the Django framework, obviously with its latest version, 6.x. All the frontend and backend elements in our project are based on this framework. The frontend assets will let you see the Tailwind CSS configuration, which allowed the development of the website's main design features. The design is based on a retro style with some added modern elements that cause the website to have the typical look that these kinds of platforms normally have.

Furthermore, all the database migrations are also available. Those are the main database tables that the platform will use to ensure that all the features the project wants to achieve are completed. Finally, you will find the current development roadmap, with signs of Work in Progress (W.I.P.) from the future features that will be implemented in the near future for this project, distributed along the source code.

1.4. Development

This project is mainly a learning workflow. All the features and code implemented are purely academic. There is no current intention to bring this website to a further step in production. Basically, it is the starting point to learn a new framework that allows the development of web pages via a programming language such as Python, a knowledge that might come in handy in our close/far future.

2. Notes about all the design decisions.

In the following section, we will visit each one of the main points from our proposed solution, and we will also see all those design decisions that have conditioned the development of this first deliverable.

2.1. Database schema.

First of all, we have the main structure of the backend: in other words, the database. Initially, we presented the schema from Figure 1, but, after receiving the professors' feedback, as well as during the implementation process, some decisions and changes were made to simplify the design, and to make everything possible, which lead to the current database version that can be seen in Figure 2.

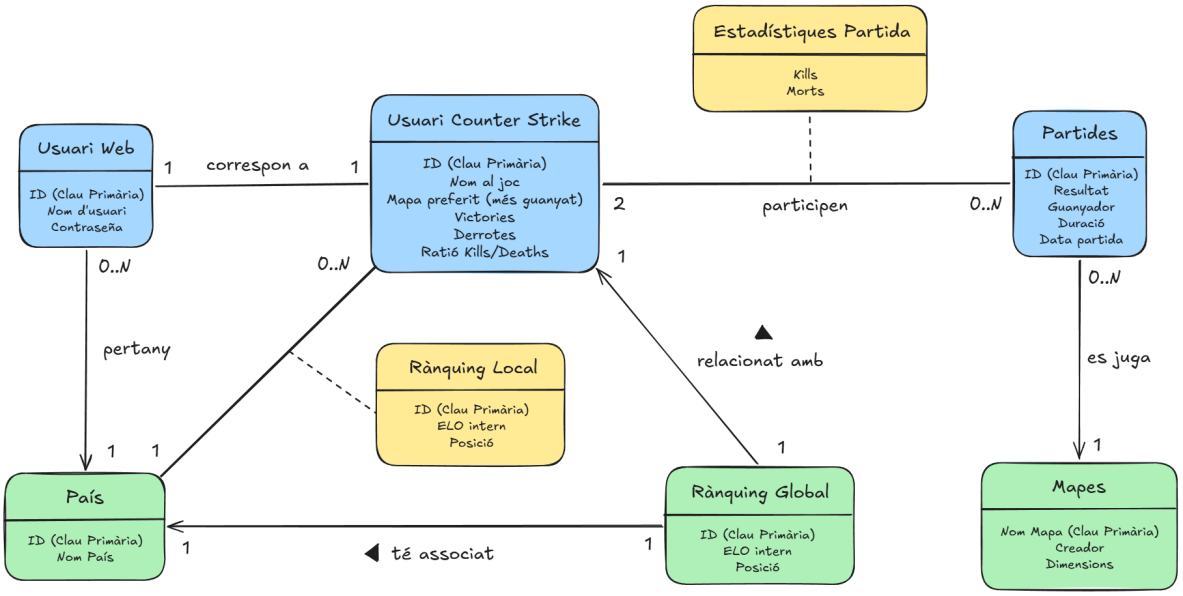

Figure 1: Initial database schema

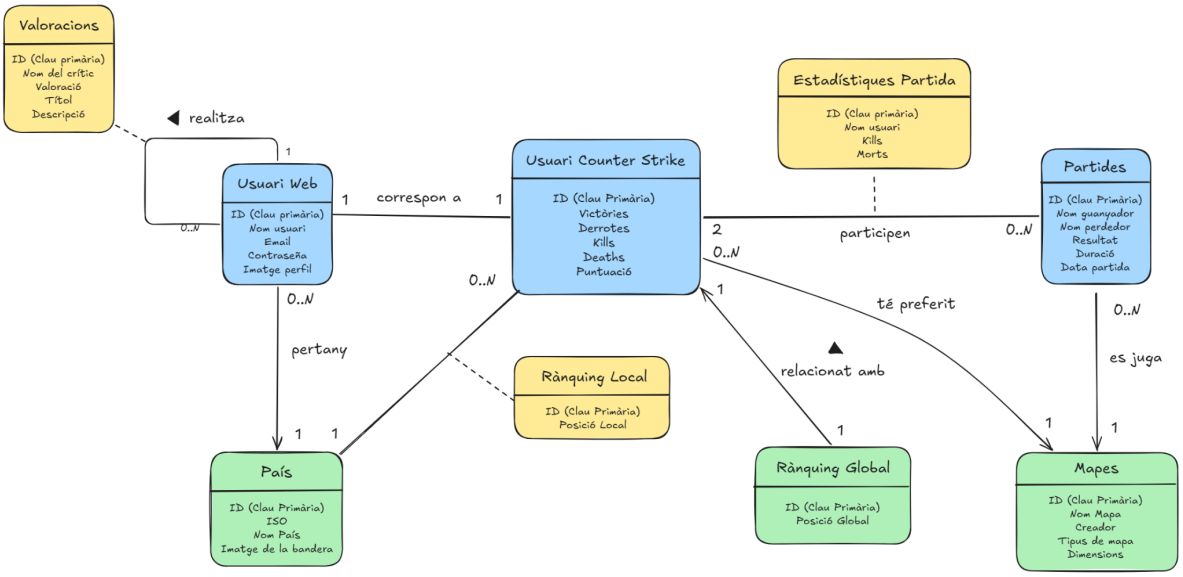

Figure 2: Updated database schema

We'll explain now the main decisions that made us update the database schema. The Map table serves as a repository for arena metadata, so we added more parameters to provide more details to each map, specifically their physical dimensions and map types, that group each map according to their size. We also added a favourite map field to the users, in order to give the maps more relevance.

The Country table manages regional identification, and, in order to make this purpose more clear, we added each country's ISO code (a code that uniquely identifies a country), as well as an image from one of the API we mentioned when we were sketching this project. This way, since we're storing these images on creation (we obtain them from the API), if the flags API of the flags disappears, the website will still be working properly.. We also decided to add a default country for the users that don't wanna provide it using the Antarctica flag from the same API as the other countries, basically for consistency.

The user management system is still divided into two layers to ensure account security and custom items tracking. The WebUser table extends Django's AbstractUser, which allows it to reuse its native authentication model, handling sensitive credentials and session management while enforcing specific validation constraints, such as restricted email domains (@gmail.com). This was changed to reduce the amount of work to do and rely more on the default Django elements. This identity is built via a one-to-one relationship with the CounterUser table, which functions as the competitive heart of the profile. It contains information about the user's lifetime wins, losses, kills and total score, gathered and accumulated from each match. We changed the user parameters to give more detailed stats as well as to provide more customization, allowing the users to have a profile picture of their own. We also added an email field in order to have more user data from their registration. This system achieves a clean separation, allowing the competitive profile to evolve independently of the web account settings.

Competitive interactions and historical records are still managed inside a relationship built between matches and performance metrics. The Match table records the details of each encounter, including the winner and the loser, the final score (new addition), its duration, and date. To capture the granular performance of each participant, the system utilizes the MatchStats, keeping the more concrete data from each user in each match better distributed, such as individual kills and deaths for every specific game. We simplified the design by storing both winner and loser instead of relying on a Many-to-Many field that was overcomplicating everything. We added the username to MatchStats since it wasn't that clear who that instance was referring to, and we also changed the way we identify every single match.

Finally, the platform still implements a ranking hierarchy. The GlobalRanking and LocalRanking tables are used to rank players, both generally and depending on their respective countries. We no longer store the country in the global relationship, since it can be fetched via other relations, following what we previously did with the users' score: we prefer to avoid duplication, even if that makes us take a longer alternative route. Some other minor changes were added to the implementation: str methods, renamed fields, reverse method references, several validators, specific names for the admin interface, but the design otherwise was mostly preserved, using mostly foreign keys or one-to-one relationships.

With professors' feedback, we also added a Review table in order to facilitate social accountability between players. There are check constraints to prevent self-rating and duplicate entries, but they're also present in other relations as well, preventing self-matches and repeated ranking positions in the same country, amongst other defined constraints. This reflexive relation will also be really useful for the second following deliverable. This is the current state of our database, but it may change in the future depending on the website's evolution and its remaining implementation (it's still missing some methods needed for further updates as well, which will be done alongside the other two deliverables).

2.2. Admin interface.

We'll now discuss a bit about the admin features. We haven't focused much on changing the visual aspects of the interface, even though we hid the group permission table since we're not going to use it for now. As for the database tables, we mostly used the same admin properties and procedures: we showed all the table attributes, except those cases where we have an object and a string, so when we delete an item but we want to keep the related one we can still show the deleted item's main attribute. This is what happens, for example, with match and both its users, and with reviewer and its review.

We also made most of the fields easily editable, even though they might not be like that in the future, in order to help debugging and allow easy changes even on related items. There are some exceptions as well, for those fields we considered immutable once created, like the match's date, or some other key or defining parameters (such as the reviewee in the review table). We also used some functions in order to achieve some of this explained behaviour, or to make a field editable only in creation, since the options given (read only) didn't allow that. We also defined a 20 page limit for the amount of tuples we'd like for each page, and stated specific ordering criteria for every single relation. We even had to overwrite some specific django functions like save, for example, in order to allow that variable-string dichotomy from before. Finally, in order to implement the desired bidirectionality from our two users, the web one and the counter one (for playing games), we decided, in the end, to not allow counter users to be directly deleted, but only through its web users, since django didn't allow both directions.

2.3. User authentication and registration.

Now that we know how the database is supposed to work and how we manage the data from the admin interface, we can explain in more detail the user authentication and registration in our platform. The process begins with the registration phase, where the platform utilizes a customized version of Django's UserCreationForm, since we're using a custom user (WebUser) instead of the base Django User in our database. This specialized form, is in charge of obtaining other websites' common credentials alongside the project-specific data, such as the user's email, username and home country.

Once the registration data is submitted, the backend handles all the credential storage. It also takes advantage of various Django security mechanisms, as it's more comfortable than implementing them from scratch. Since at the moment only the WebUser is needed, we're not creating the corresponding CounterUser at the same time, though it's a feature that will be eventually added, along with all the currently unimplemented functionality, that is mostly linked with the matches and both player rankings.

The authentication mechanism (or login flow), is built upon a secure session-based architecture. When a player attempts to access the website, the platform utilizes the AuthenticationForm to verify the provided credentials against the stored hashes. Given a successful match, Django's login utility initiates a secure session and delivers a CSRF (Cross-Site Request Forgery) token to the user. That single action is crucial, since it grants our users protection against malicious third-party interferences.

The platform uses the AbstractUser class to extend the default Django User model, which, unlike the aforementioned, allows us to add new fields on top of the existing ones. This way, the platform gains access to built-in permission systems and groups, and the administrative interface can manage users alongside other relations effectively. At the same time, this also lets us enforce another essential rule: players can only modify their own profile data, preventing them from modifying other users' profiles.

2.4. Docker orchestration using docker-compose.

In order to deploy our website and get a public URL, we're using Docker and its orchestration through Docker-Compose, as requested in this first deliverable. This ensures portability, scalability, and easy deployment, key properties that we want our project to have. This application is encapsulated in a lightweight Python 3.13-slim base image, which provides a secure and minimal runtime environment.

Another key factor of our building process is that this project uses uv to manage all its dependencies. Uv is a well-known high-performance Python package installer and resolver. Following our teachers' advice, by using *uv sync* specifically in our project, after setting everything up, we can be sure that the dependencies will all be locked and installed properly and efficiently. This way, we're also enabling Docker's layer caching and significantly reducing build times of the project, as well as allowing us to spare ourselves any dependency-related inconvenience that could have arisen while implementing.

The Docker-Compose configuration serves as the orchestration layer, defining how the application services are going to interact with the host system. It maps the internal container port 8000 to the host machine, allowing access via the local browser. Another feature we can find in this setup is the usage of volumes. By mounting the local project directory into the /app folder within the container, the system enables live-reloading, which means that any code changes on your machine are reflected instantly inside the running container, without requiring a full image rebuild, which, proved to be really useful.

To handle the transition from a static image to a running service, the project makes use of a dedicated entrypoint script (*entrypoint.sh*). This script acts as the automated "boot sequence" for the application. It executes three vital tasks: to begin with, it runs database migrations to ensure that the relational schema is up-to-date. Then, it also executes collectstatic to gather all frontend assets (Tailwind CSS, images, and retro UI elements) and put them into a single directory so the server can access it, and finally, it launches Gunicorn, a production-grade WSGI HTTP server that has been implemented using the specific Django documentation that appears when you execute the command *python manage.py runserver* in the development environment. This orchestration facilitates future expansions of our platform, like adding dedicated database containers (PostgreSQL), with minimal architectural friction.

2.5. Adhere to the 12-factor guidelines.

In order to ensure that the web application we're developing is portable, scalable, and easy to deploy, the Strike 1v1 project website tries to adhere as much as possible to the famous 12-factor guidelines.

1. **Codebase:** We're only maintaining a single version-controlled repository on GitHub. Every deployment, no matter if it belongs to either staging or production, will always come from the same repository (it's not a distributed system), ensuring consistency across all environments.

2. **Dependencies:** Our Strike 1vs1 project, explicitly declares all its Python dependencies, as explained previously, by using the widely used uv package. The project contains two specific files, which are *pyproject.toml* and *uv.lock,* that make this possible. By isolating all these dependencies from the host system, we can be sure it won't rely on system-wide packages.

3. **Config:** Configuration that varies between environments (such as the Django SECRET_KEY) is always stored in environment variables, allowing the application to be deployed in different contexts without modifying the source code, and without exposing them to the general public.

4. **Backing Services:** We're currently not following this factor for every service we rely on. Our project is still using the default database file that comes with Django (sqlite file), since this, in particular, wasn't the main goal of this deliverable nor something needed in the early stages. In the near future, though, we'll most likely change it to a PostgreSQL service database (so we can connect using URLs), but for this delivery, we'll be using the default SQLite database.

5. **Build, Release, Run:** Our project also tries to separate all these 3 stages. The Build stage is handled by Docker (it creates the image), the Release stage combines the image with our environment's config, and finally the Run stage is executed by Gunicorn inside the container.

6. **Processes:** The website is executed as a stateless process, as it should be. Any persistent data (such as the player's score (ELO), its wins, losses, deaths, kills, or other statistics, or the

whole match history, amongst other attributes) is always stored inside the database, never on the website itself. With the future staged migration to a PostgreSQL database, we also expect to follow this guideline better, as we're not using backing services for our DB at the moment.

7. **Port Binding:** The website is completely self-contained.It doesn't rely on any external web server. It exports its service by binding directly to port 8000, which is then mapped via Docker.

8. **Concurrency:** The architecture is mainly designed for horizontal scaling, rather than for the vertical variant. This is because the website has been designed to be stateless. This way, we can handle increased traffic by simply running more instances of the "web" containers instead of using one huge instance to manage everything. We're also planning to use this strategy for the game containers: each game will be played in a separate container, allowing concurrency.

9. **Disposability:** The app is optimized for fast startup and graceful shutdown. Using uv and a lightweight Python image that contains only what we need, we allow the container to be ready in seconds. This makes all deployments and scalings fast and safe, just as this factor states.

10. **Dev/Prod Parity:** By using Docker-Compose, we ensure that our development environment is an exact replica of our production environment. This avoids the "works on my machine" issue this factor talks about by using the same OS, Python version, and libraries everywhere.

11. **Logs:** The application treats logs as event streams. Basically, the app writes everything to the standard output, following this factor's guidelines. This allows Docker to collect and route logs according to our environment's needs: this way, it's never an app responsibility in the first place. In Render, our chosen deploy service, there is also a section where you can collect and store all this information, fulfilling the entirety of what this second-to-last factor stands up for.

12. **Admin Processes:** One-off administrative tasks, such as the database migrations (handled by python manage.py migrate), are run in the same codebase and configuration from the main app processes. Since we have only one codebase and config, it applies to all of those.

We can see that we currently follow most of them, the most important ones. It might not follow them all at the moment, but, we've compromised on following the remaining ones for the next deliverables.

2.6. Web pages to display and allow browsing application data.

With all the backend stuff already mentioned, we can move forward to the frontend and what the user will see when it enters the website. In this section, we'll basically describe the main pages that have been implemented at the moment, and all the information they are trying to show to our future users.

First of all, we'll mention that we have been using both Figma and Stitch to create prototypes of the website, in order to make the styling part easier for us, as well as providing a minimum responsive website and not rely entirely on the implementation of raw HTML and CSS. We've never worked with these types of files, and this is useful to learn about how they work and what their main function is in a website environment. In addition, we've been using Tailwind CSS to efficiently manage all the colors from the website, and a *main.css* file. Both files are available in the static folder inside our repository.

The project define two base templates in the base folder. They try to facilitate the implementation of future web pages. These ones are the *base.html* and *base_legal.html*. The first one is the HTML that contains the header and footer HTMLs. This way, all the web pages share them, and they only need to focus on the main content of these ones. The second one is just a little extension of the *base.html* and is used in the typical web pages of Terms of Service, Privacy Policy, and Cookie Policy to have the same structure.

Then there are the *header.html* and *footer.html* in the components folder. As expected, they are the HTML that implements the header and footer of the website. Just mention that in the header, when the user has successfully completed the login phase of the website, the landing page disappears and it is substituted by the dashboard page of the current user. In both files there is a little script of JavaScript to implement two cool features. On one hand, in the *header.html*, it allows a menu display to access the user profile page (that currently is not available (W.I.P.)) and log out from the website. On the other hand, *footer.html* has just a clock to display the current time for the user.

Furthermore, there are the Terms of Service, Privacy Policy, and Cookie Policy HTML in the legal folder. All of them use the *base_legal.html* and they just need to focus on writing their own content.

Arriving at the end, we have the pages folder. There we have the main HTML for the pages that can be browsed on the website. We are going to comment on them briefly.

1. **landing.html:** This one contains the main page that users will encounter once they enter the website and have not been logged in before. It just displays information about what Strike 1v1 is and what the main objectives are.

2. **dashboard.html:** Once the user is logged in, the landing page disappears for them, and a dashboard appears. There the player can see basic information about his user name, profile picture, region, ELO points, and statistical stuff such as wins, losses, and win rate.

3. **play.html:** This is the main page that allows the users to start the matchmaking and start playing in the web browser. But this feature is still under development, and when users try to start a match, it appears a server notification indicating that the feature is still in progress.

4. **leaderboard.html:** This page allows the user to visit the different leaderboards around the different regions. The positions of the players are based on their current ELO, and if the user is on the leaderboard page, he is marked in a special color to be easily seen.

5. **login.html and register.html:** These two pages are basically the HTML related to the login and register pages. They used the Django structure to secure the data integrity and validate users.

3. Distribution of marks

The development of this project has been a very organized collaborative effort, where all tasks were designated according to our technical specialization and knowledge, having our interests in mind as well when distributing the workload, in order to ensure the effectiveness of every specific project goal.

Pol Ortega Fernàndez took care of the whole database architecture, taking responsibility for the design ideas and diagram building and the implementation of the relational model in Django, as well the configuration of the administrative interface, and the final authentication logic from our website.

Marc Pujol Navajo's role was the "Front End" designer and DevOps workflows, managing the Dockerization of the services, the orchestration via Docker-Compose, and the implementation of the web interface based on the team's design requirements, as well as being in charge of the whole Git project, and also adding the main workflow deployment to Render to give our website a public URL.

Andreu Puig-gròs Munté provided critical support during the initial database design phase as well as developing the foundations of the structure for user authentication. Also, outside this deliverable, he managed to make a functionable server for the 1v1 matches, extracting and processing the data for the matches in a compatible format for the DB, handling basically everything that was server related.

Despite these different areas of focus, all our design decisions were made collectively. Since every member fulfilled their designated responsibilities and contributed to the essential elements of the final application, we propose, as a team, an equal grade for all the members, because we think the amount of work that each of us has done is similar. Therefore we believe we deserve to have the same grade.